Decoder Trees and Kisielewicz's Identity: Equations 5093 and 5107 Are Austin Laws

KEIO*

28 October 2026

Abstract

An *Austin law* is an equational law that has non-trivial infinite models but no non-trivial finite models. Kisielewicz (1997) showed that the order-five law

$$x \approx y \diamond \big(y \diamond \big(y \diamond (x \diamond (z \diamond y))\big)\big)$$

(Equation 5093 in the numbering of the Equational Theories Project) has no non-trivial finite model, and the existence of a non-trivial infinite model has remained open. The question is listed among the open questions of the Equational Theories Project, and the automated trace-tree search of 2026 did not settle it. We answer the question: Equation 5093 is an Austin law. More generally, for every $k \geq 3$ the two laws

$$\mathsf{E}_k\colon\ x \approx L_y^k\big(x \diamond (z \diamond y)\big), \qquad \mathsf{G}_k\colon\ x \approx L_y^k\big(z \diamond (x \diamond y)\big), \qquad L_y(t) := y \diamond t,$$

are Austin laws, while for $k = 1, 2$ (Equations 69, 507, 83, 521) they are equivalent to $x \approx y$, so the threshold $k = 3$ is sharp. Since G_3 is Equation 5107, another order-five law whose infinite models were unknown, we obtain four new Austin laws of order five: 5093 and 5107 and their duals 41179 and 40951. The models are *decoder-tree magmas*. A product of binary trees contracts exactly when its right factor is a tower of $k-1$ copies of the left factor over a core that is *semantically decodable*. Decodability and the companion notion of a *key* are defined by a well-founded mutual recursion with stratified negation. We prove that the normal decoder trees form the free algebras of these laws, which also solves their word problems. The proofs are short and self-contained. An appendix contains a verification program. It checks the laws on more than $6 \cdot 10^6$ instances, including $6 \cdot 10^4$ instances aimed at the two contracting decoding clauses, and checks every structural lemma on $1.2 \cdot 10^5$ further instances.

Contents

1 Introduction **2**
1.1 Austin laws and Kisielewicz's question 2
1.2 Results 2
1.3 The method: decoder trees 3
1.4 Relation to earlier constructions 4

2 Preliminaries **4**

3 The collapse for $k \leq 2$ **4**

*Research draft prepared with AI assistance. All proofs are written out in full for independent checking, and a self-contained verification program is reproduced in Appendix A. All proofs were also re-checked case by case, with independently written code, by a separate AI review agent, which found no mathematical errors. The manuscript has not been externally peer reviewed or certified by a proof assistant.

4 Decoder trees **5**
4.1 Trees 5
4.2 Decodability, firing, keys and owners 5

5 Structural lemmas **6**

6 Proof of the main theorem and corollaries **9**

7 Discussion **10**
7.1 How the construction was found 10
7.2 The decoder-tree principle 10

A Verification program **11**

1 Introduction

1.1 Austin laws and Kisielewicz's question

A *magma* is a set with a binary operation $\diamond$, and an *equational law* is a universally quantified identity $w_1 \approx w_2$ between magma words. Every law is satisfied by the one-element magma; a law is *trivial* if this is its only model up to isomorphism, equivalently if it implies $x \approx y$. Austin [1] constructed the first law that has non-trivial models but no non-trivial *finite* ones; such laws are now called *Austin laws*. Austin also showed that every law in at most two variables that has a non-trivial model has a non-trivial finite one [2]. Kisielewicz gave much shorter Austin laws, first of order six [6] and then of order five [7], and reported a computer search showing that no law of order at most four is an Austin law [7].

In the same paper Kisielewicz pointed to the order-five law

$$x \approx y \diamond (y \diamond (y \diamond (x \diamond (z \diamond y)))) \tag{E5093}$$

and showed that it has no non-trivial finite model. Whether it has a non-trivial infinite model, that is, whether it is an Austin law, was left open. The Equational Theories Project (ETP) [4, 5] numbers this law 5093. It lists the question among the open problems in the conclusions of its final report [4], and its blueprint records that whether 5093 is an Austin law is an open question [5]. Law 5093 is one of 96 order-five laws whose finite models are trivial but whose infinite models were unknown at the end of the ETP [5, chapter "Order 5 Austin laws"]; another is Equation 5107,

$$x \approx y \diamond (y \diamond (y \diamond (z \diamond (x \diamond y)))). \tag{E5107}$$

In September 2026, Zhao, Wu, Yang and Miao [9] used automatically synthesised trace-tree magmas to certify 32 of these 96 laws as Austin, 28 of them for the first time. The other four had already appeared among the six positives of the ALPS benchmark [3], so 34 of the 96 were classified before the present work. Neither 5093 nor 5107, nor their duals, was among them: the public result files of [9] record "no candidate" for 5093 and 5107, and their same-resource ATP baselines exhausted their resources on both.

1.2 Results

For $k \geq 1$ consider the two laws, of order $k+2$,

$$\mathsf{E}_k\colon \quad x \approx L_y^k\big(x \diamond (z \diamond y)\big), \tag{1}$$

$$\mathsf{G}_k\colon \quad x \approx L_y^k\big(z \diamond (x \diamond y)\big), \tag{2}$$

where $L_y(t) := y \diamond t$. Thus E_3 is (E5093) and G_3 is (E5107). In ETP numbering, $\mathsf{E}_1, \mathsf{E}_2, \mathsf{G}_1, \mathsf{G}_2$ are Equations 69, 507, 83 and 521. We pose the following problem and solve it completely.

Question 1.1 (Classification problem)**.** For which $k \geq 1$ is E_k an Austin law, and for which k is G_k?

Theorem 1.2 (Main theorem)**.** *Let $k \geq 3$ and let X be a non-empty set. On the set $\mathcal{T}(X)$ of finite binary trees with leaves in X there are explicit operations $\diamond_\lambda$ and $\diamond_\omega$, the* decoder-tree operations *of Definition 5.5, such that $(\mathcal{T}(X), \diamond_\lambda) \models \mathsf{E}_k$ and $(\mathcal{T}(X), \diamond_\omega) \models \mathsf{G}_k$. In each case the submagma $\mathcal{N}(X)$ of normal trees (Definition 6.2) is the free algebra on X of the corresponding law.*

Corollary 1.3. *For every $k \geq 3$ the laws E_k and G_k are Austin laws. In particular Equations 5093 and 5107 are Austin laws, and so are their duals, Equation 41179, $x \approx ((((y \diamond z) \diamond x) \diamond y) \diamond y) \diamond y$, and Equation 40951, $x \approx ((((y \diamond x) \diamond z) \diamond y) \diamond y) \diamond y$. This answers Kisielewicz's question [7, 4].*

Corollary 1.4. *For every $k \geq 3$ the word problems of E_k and G_k are decidable: an identity $s \approx t$ follows from the law if and only if s and t evaluate to the same normal tree in $\mathcal{N}(V)$, where V is the set of variables.*

Proposition 1.5 (The threshold is sharp)**.** *The laws $\mathsf{E}_1, \mathsf{E}_2, \mathsf{G}_1, \mathsf{G}_2$ are trivial: each implies $x \approx y$.*

So both families have the same dichotomy, which answers Question 1.1. For $k \leq 2$ the law is trivial, in line with Kisielewicz's computer search, since these laws have order at most four. For every $k \geq 3$ the law is Austin, and its free algebras are described explicitly.

1.3 The method: decoder trees

The construction rests on one observation, which applies to every model M of E_k or G_k. Call $w = c \diamond a$ an *a-key* with *owner* c. Then E_k says that k left multiplications by a recover x from $x \diamond w$, the element x "locked" with an a-key. G_k says the same about recovering the owner x from $z \diamond w$, where $w = x \diamond a$. In the absolutely free magma we therefore keep every product formal, except that a product contracts exactly at the k-th unlocking step:

$$a \diamond \underbrace{\langle a, \langle a, \cdots \langle a, u \rangle \rangle \rangle}_{k-1} \;=\; x \quad \text{whenever } u \text{ is decodable by } a \text{ with value } x.$$

Being an a-key and being decodable are themselves defined through the operation, so the definition is self-referential. The key steps of the paper are:

(i) to unfold decodability into three explicit clauses $D_\alpha, D_\beta, D_\gamma$ (Definition 4.1), which form a well-founded recursion on size; the two laws differ only in the value that a clause returns: the locked element for E_k, the owner of the key for G_k;

(ii) to prove three structural facts: decoded values are proper subtrees (*subterm property*), at most one clause applies (*uniqueness*), and neither a itself nor an a-key is ever decodable (*vacuity*);

(iii) to deduce from vacuity that none of the first $k-1$ unlocking steps contracts prematurely. The laws then follow in a few lines.

The hypothesis $k \geq 3$ is a standing assumption; for instance, it makes clause (β) size-decreasing. It is essential at two points: it forces the left factor of an unlocking product to be determined by the key (Lemma 5.6), and it separates towers from keys (Lemmas 5.9 and 5.10). For $k = 2$ both fail. By Proposition 1.5, any such construction must break down for $k \leq 2$.

1.4 Relation to earlier constructions

Self-referential normal forms have been used before. The ETP's description of the free magma for Equation 677 [5, chapter "Equation 677"] contracts $x \diamond y$ to y_L when $y = \langle y_L, (x \diamond y_L) \diamond x \rangle$, a single syntactic pattern with a recursive side condition. The trace-tree magmas of [9] default to pairing and take finitely many exceptional outputs, defined by positive Horn clauses with a least-fixed-point semantics. Our operations differ in two ways. First, the firing condition is a *semantic* predicate, decodability, defined by mutual recursion with the key predicate rather than by a fixed finite set of patterns. Second, that recursion uses negation: a formal pair $\langle c, a \rangle$ counts as a key only if $c \diamond a$ does *not* contract. The negation is stratified by size, so the recursion remains well-founded. A naive Knuth–Bendix completion [8] of (E5093) did not terminate in our experiments (Remark 7.1). The decoder-tree operation can be viewed as a finite recursive description of the completed rewriting system.

Organisation

Section 2 collects elementary facts, including the absence of finite models. Section 3 proves Proposition 1.5. Section 4 defines decoder trees, Section 5 proves the structural lemmas, and Section 6 proves Theorem 1.2 and its corollaries. Section 7 discusses the method and further questions, and Appendix A contains the verification program.

2 Preliminaries

For a magma $(M, \diamond)$ and $a \in M$ write $L_a(t) = a \diamond t$ and $R_a(t) = t \diamond a$.

Proposition 2.1. *Let $k \geq 1$ and let $(M, \diamond)$ satisfy E_k or G_k. Then:*

(a) *every L_y is surjective;*

(b) *every R_w is injective, i.e. M is right-cancellative;*

(c) *if M is finite, then $|M| = 1$.*

Proof. (a) In both laws, $x = L_y(\,\cdot\,)$ for every x.

(b) For G_k, the law says $L_y^k L_z R_y = \mathrm{id}$ for all y, z, so each R_y is injective. For E_k, let $w \in M$. By (a), $w = s \diamond t$ for some s, t, and E_k with $y = t$ and $z = s$ gives $L_t^k(x \diamond w) = x$ for all x. So $L_t^k R_w = \mathrm{id}$ and R_w is injective.

(c) For E_k this is Kisielewicz's argument [7], formalised in the ETP [5]. In a finite model each L_y is a bijection by (a), so $x \diamond (z \diamond y) = L_y^{-k}(x)$, which does not depend on z. Since L_x is injective, $z \diamond y =: c(y)$ does not depend on z. Then $x \diamond (z \diamond y) = x \diamond c(y) = c(c(y))$ does not depend on x, and E_k gives $x = L_y^k(c(c(y)))$ for every x.

For G_k, again $z \diamond (x \diamond y) = L_y^{-k}(x)$ does not depend on z. By (b) and finiteness, R_y is a bijection, so every $t \in M$ has the form $x \diamond y$. Hence $z \diamond t =: f(t)$ does not depend on z. Then $x \diamond y = f(y)$ and $z \diamond (x \diamond y) = f(f(y))$, and G_k gives $x = f^{k+2}(y)$ for every x. □

So for every k, a non-trivial model of E_k or G_k must be infinite, and each of these laws is an Austin law if and only if it has some non-trivial model.

3 The collapse for $k \leq 2$

Proof of Proposition 1.5. Throughout, a, b, c denote arbitrary elements of a model.

E_1*:* $y \diamond (x \diamond (z \diamond y)) \approx x$. By E_1 with $(x, y, z) = (c, b, a)$ we have $b \diamond (c \diamond (a \diamond b)) = c$. By E_1 with $(x, y, z) = (b,\ a \diamond b,\ c)$, $b = (a \diamond b) \diamond \big(b \diamond (c \diamond (a \diamond b))\big) = (a \diamond b) \diamond c$. So $(a \diamond b) \diamond c = b$ for all a, b, c. Hence, by E_1 with $y = a \diamond b$, $x = (a \diamond b) \diamond \big(x \diamond (z \diamond (a \diamond b))\big) = b$ for all x, b.

E_2: $y \diamond (y \diamond (x \diamond (z \diamond y))) \approx x$. Put $Y := a \diamond (b \diamond c)$. By E_2 with $(x, y, z) = (a, c, b)$ we have $c \diamond (c \diamond Y) = a$. By E_2 with $(x, y, z) = (c, Y, c)$, $c = Y \diamond (Y \diamond (c \diamond (c \diamond Y))) = Y \diamond (Y \diamond a)$. Then E_2 with $(x, y, z) = (Y, a, Y)$ gives $Y = a \diamond (a \diamond (Y \diamond (Y \diamond a))) = a \diamond (a \diamond c)$. So $a \diamond (b \diamond c) = a \diamond (a \diamond c)$ does not depend on b. Hence, for all x, x', y, z,

$$x = y \diamond \big(y \diamond (x \diamond (z \diamond y))\big) = y \diamond \big(y \diamond (x' \diamond (z \diamond y))\big) = x'.$$

G_1: $y \diamond (z \diamond (x \diamond y)) \approx x$. By G_1 with $(x, y, z) = (a, b, c)$ we have $b \diamond (c \diamond (a \diamond b)) = a$. By G_1 with $(x, y, z) = (c,\ a \diamond b,\ b)$, $c = (a \diamond b) \diamond \big(b \diamond (c \diamond (a \diamond b))\big) = (a \diamond b) \diamond a$. So c does not depend on c.
G_2: $y \diamond (y \diamond (z \diamond (x \diamond y))) \approx x$. Put $Y := a \diamond (b \diamond c)$. By G_2 with $(x, y, z) = (b, c, a)$ we have $c \diamond (c \diamond Y) = b$. By G_2 with $(x, y, z) = (c, Y, c)$, $c = Y \diamond (Y \diamond (c \diamond (c \diamond Y))) = Y \diamond (Y \diamond b)$. Then G_2 with $(x, y, z) = (Y, b, Y)$ gives $Y = b \diamond (b \diamond (Y \diamond (Y \diamond b))) = b \diamond (b \diamond c)$. So $a \diamond (b \diamond c) = b \diamond (b \diamond c)$ for all a, b, c. By Proposition 2.1(a), every t can be written $t = b \diamond c$ for any given b; hence $a \diamond t = b \diamond t$ for all a, b, t, i.e. the product does not depend on its left factor. In particular $x \diamond y = x' \diamond y$, and G_2 gives $x = y \diamond (y \diamond (z \diamond (x \diamond y))) = y \diamond (y \diamond (z \diamond (x' \diamond y))) = x'$. □

Remark 3.1. These derivations are the first critical pairs of the rewrite rules $L_y^k(x \diamond (z \diamond y)) \to x$ and $L_y^k(z \diamond (x \diamond y)) \to x$. For $k = 3$ the analogous critical pairs do not collapse. They generate new rules instead, and in our experiments the completion did not terminate (Remark 7.1).

4 Decoder trees

From now on $k \geq 3$ is fixed, X is a non-empty set, and $\theta \in \{\lambda, \omega\}$ is a fixed *type*. Type λ ("locked element") will produce models of E_k, and type ω ("owner") models of G_k.

4.1 Trees

Let $\mathcal{T} = \mathcal{T}(X)$ be the set of finite rooted binary trees with leaves labelled by X, i.e. the absolutely free magma on X. Its elements are the leaves $g \in X$ and the formal pairs $\langle s, t\rangle$ with $s, t \in \mathcal{T}$. For a pair $p = \langle s, t\rangle$ write $p_L = s$ and $p_R = t$. The *size* $|t|$ is the number of nodes: $|g| = 1$ and $|\langle s, t\rangle| = |s| + |t| + 1$. Write $s \sqsubset t$ if s is a *proper* subtree of t, and let $\mathrm{Sub}^-(t)$ be the set of proper subtrees of t. If $s \sqsubset t$ then $|s| < |t|$, and $t \notin \mathrm{Sub}^-(s)$.

For $c, u \in \mathcal{T}$ and $j \geq 0$ define the *c-tower of height j over u*:

$$c^{[0]}u := u, \qquad c^{[j+1]}u := \langle c, c^{[j]}u\rangle.$$

Given c, p and j there is at most one t with $p = c^{[j]}t$. If $j \geq 1$ then $|c^{[j]}u| = j(|c| + 1) + |u|$, and both c and u are proper subtrees of $c^{[j]}u$.

4.2 Decodability, firing, keys and owners

Definition 4.1 (Decoder-tree data of type θ)**.** By recursion on $n = |a| + |u|$ we define a set $D(a, u) \subseteq \mathcal{T}$ for all $a, u \in \mathcal{T}$. Suppose $D(a', u')$ is already defined whenever $|a'| + |u'| < n$. We use the following abbreviations; each refers to D only at arguments of total size $< n$, as indicated in brackets.

- *Firing.* For $c, p \in \mathcal{T}$ let $F(c, p)$ hold if $p = c^{[k-1]}t$ for some t with $D(c, t) \neq \emptyset$. $[|c|+|t| < |p|]$
- *Special key.* For a pair a with $a = a_L^{[k-1]}t$ put $\kappa(a) := D(a_L, t)$; otherwise $\kappa(a) := \emptyset$. $[|a_L| + |t| < |a|]$
- *Keys and owners.* $K(a) := \{\langle c, a\rangle : c \in \mathcal{T},\ \neg F(c, a)\} \cup \kappa(a)$. The elements $\langle c, a\rangle$ of the first set are *generic keys*; they are larger than a. The elements of $\kappa(a)$ are *special keys*; they are smaller than a (Lemma 5.1). The *owner* of a key $w \in K(a)$ is $o_a(w) := c$ if $w = \langle c, a\rangle$ with $\neg F(c, a)$, and $o_a(w) := a_L$ otherwise, i.e. if w is special.

- *Output.* For $x \in \mathcal{T}$ and $w \in K(a)$ put $\theta_a(x,w) := x$ if $\theta = \lambda$, and $\theta_a(x,w) := o_a(w)$ if $\theta = \omega$.

Now set $D(a,u) := D_\alpha(a,u) \cup D_\beta(a,u) \cup D_\gamma(a,u)$, where:

(α) $D_\alpha(a,u) = \{\theta_a(u_L, u_R)\}$ if u is a pair with $u_R \in K(a)$ and $\neg F(u_L, u_R)$; otherwise $D_\alpha(a,u) = \emptyset$.

(β) $D_\beta(a,u) = \{a_L\}$ if a is a pair with $a = a_L^{[k-2]}s$ for some s, $\neg F(a_L, a)$, and $u \in D(a_L, s)$; otherwise $D_\beta(a,u) = \emptyset$. $[|a_L| + |s| < |a|]$

(γ) $D_\gamma(a,u)$ is the set of all $\theta_a(x,v)$ for which there are $v \in \kappa(a)$ and s with $v = x^{[k-1]}s$, $|x| + |s| < n$, and $u \in D(x,s)$.

In (α), deciding whether $u_R \in K(a)$, and computing the owner, needs $F(c,a)$ and $\kappa(a)$; these refer to D at total size $< |a|$. Deciding $F(u_L, u_R)$ refers to total size $< |u|$. So the recursion is well-founded, and D, F, κ, K and o_a are well defined on all of $\mathcal{T}$. By Lemma 5.1 the two cases in the definition of the owner are disjoint, since generic keys are larger than a and special keys are smaller. All these notions depend on θ; since θ is fixed we suppress it from the notation. The guard $|x| + |s| < n$ in (γ) is always satisfied (Remark 5.2); it is included only to make the recursion manifestly well-founded.

The intended meaning, made precise in Lemma 5.6, is: $K(a)$ is the set of right multiples $c \diamond a$, and $o_a(c \diamond a) = c$; $D(a,u)$ collects the values $\theta_a(x,w)$ over all ways of writing $u = x \diamond w$ with $w \in K(a)$; and $F(c,p)$ says that $c \diamond p$ contracts. The three clauses list the three ways in which $u = x \diamond w$ can happen. Either $x \diamond w$ is the formal pair $\langle x, w\rangle$ (α), or $x \diamond w$ contracts with a generic key, which forces $x = a_L$ and $w = \langle a_L, a\rangle$ (β), or $x \diamond w$ contracts with the special key $w \in \kappa(a)$ (γ). In case (β) the output is $\theta_a(a_L, \langle a_L, a\rangle) = a_L$ for both types.

5 Structural lemmas

Lemma 5.1 (Subterm property)**.** *For all $a, u \in \mathcal{T}$: $D(a,u) \subseteq \mathrm{Sub}^-(a) \cup \mathrm{Sub}^-(u)$ and $\kappa(a) \subseteq \mathrm{Sub}^-(a)$.*

Proof. Induction on $|a| + |u|$. If $a = a_L^{[k-1]}t$ then, by induction, $\kappa(a) = D(a_L, t) \subseteq \mathrm{Sub}^-(a_L) \cup \mathrm{Sub}^-(t) \subseteq \mathrm{Sub}^-(a)$. Owners are proper subtrees of their keys or equal to a_L, so every value of θ_a that occurs is one of u_L, $(u_R)_L$, a_L or v_L with $v \in \kappa(a)$. These lie in $\mathrm{Sub}^-(u)$, $\mathrm{Sub}^-(u)$, $\mathrm{Sub}^-(a)$ and $\mathrm{Sub}^-(a)$ respectively. □

Remark 5.2. In (γ), $x, s \sqsubset v \sqsubset a$, so $|x| + |s| < |v| < |a| \leq n$. The guard is therefore automatic.

Lemma 5.3 (Keys)**.** *Let $a \in \mathcal{T}$.*

(a) *If $F(c,a)$ then a is a pair and $c = a_L$.*

(b) *$\kappa(a) \neq \emptyset$ if and only if $F(a_L, a)$.*

(c) *Every generic key of a properly contains a, and every special key is a proper subtree of a. In particular $a \notin K(a)$, and every key w of a with $w \sqsubset a$ lies in $\kappa(a)$.*

Proof. (a) $F(c,a)$ means $a = c^{[k-1]}t$ with $k - 1 \geq 1$, so $a = \langle c, \cdots\rangle$. (b) Both sides say that $a = a_L^{[k-1]}t$ with $D(a_L, t) \neq \emptyset$. (c) follows from the definition of $K(a)$ and Lemma 5.1. □

Lemma 5.4 (Uniqueness)**.** *For all $a, u \in \mathcal{T}$, $|D(a,u)| \leq 1$. Consequently $|\kappa(a)| \leq 1$.*

Proof. Induction on $|a| + |u|$. Each of D_α and D_β has at most one element. By induction $|\kappa(a)| \le 1$, and in (γ) the tree $x = v_L$ is determined by v; so $|D_\gamma| \le 1$. It remains to show that at most one of the three clauses applies. The argument uses only the conditions of the clauses, not their outputs, so it is the same for both types.

(β) and (γ). If (β) applies then $\neg F(a_L, a)$, so $\kappa(a) = \emptyset$ by Lemma 5.3(b), and (γ) does not apply.

(α) and (β). Suppose both apply. By (β), $u \in D(a_L, s)$ with $a_L, s \sqsubset a$, so $u \sqsubset a$ by Lemma 5.1. By (α), $u_R \in K(a)$, and $u_R \sqsubset u \sqsubset a$. By Lemma 5.3(c), $u_R \in \kappa(a)$, so $\kappa(a) \neq \emptyset$ and $F(a_L, a)$ holds. This contradicts $\neg F(a_L, a)$ in (β).

(α) and (γ). Suppose both apply, and let $\kappa(a) = \{v\}$, $v = x^{[k-1]}s$, $u \in D(x, s)$. Since $x, s \sqsubset v$, Lemma 5.1 gives $u \sqsubset v$, so $u_R \sqsubset v \sqsubset a$. By (α) and Lemma 5.3(c), $u_R \in \kappa(a) = \{v\}$, i.e. $u_R = v$. This contradicts $u_R \sqsubset v$. □

Definition 5.5 (The decoder-tree operation)**.** For $a, p \in \mathcal{T}$ define

$$a \diamond p := \begin{cases} \text{the unique element of } D(a,t) & \text{if } F(a,p),\ p = a^{[k-1]}t, \\ \langle a, p\rangle & \text{otherwise.} \end{cases}$$

This is well defined by Lemma 5.4, and by Lemma 5.1

$$a \diamond p \in \{\langle a, p\rangle\} \cup \mathrm{Sub}^-(p) \qquad \text{for all } a, p. \tag{3}$$

In particular $a \diamond p = \langle a, p\rangle$ if and only if $\neg F(a, p)$. We write $\diamond_\lambda$ and $\diamond_\omega$ for the operations of type λ and ω, and $\mathcal{T}_k^\theta(X) := (\mathcal{T}(X), \diamond_\theta)$.

Lemma 5.6 (Semantics)**.** *For all $a, u \in \mathcal{T}$:*

(a) *$K(a) = \{c \diamond a : c \in \mathcal{T}\}$, and $o_a(c \diamond a) = c$ for every c;*

(b) *$D(a, u) = \{\theta_a(x, w) :\ w \in K(a),\ x \in \mathcal{T},\ x \diamond w = u\}$.*

Consequently,

$$D(a,u) = \{x :\ \exists c\ \ x \diamond (c \diamond a) = u\} \ \text{ for } \theta = \lambda, \qquad D(a,u) = \{c :\ \exists z\ \ z \diamond (c \diamond a) = u\} \ \text{ for } \theta = \omega.$$

Proof. (a) Let $c \in \mathcal{T}$. If $\neg F(c, a)$ then $c \diamond a = \langle c, a\rangle$ is a generic key with owner c. If $F(c, a)$ then $c = a_L$ by Lemma 5.3(a), and $c \diamond a$ is the element of $D(a_L, t) = \kappa(a)$, a special key with owner $a_L = c$. Conversely, a generic key $\langle c, a\rangle$ requires $\neg F(c, a)$, and then $\langle c, a\rangle = c \diamond a$. If $v \in \kappa(a)$ then $F(a_L, a)$ holds and $v = a_L \diamond a$.

(b) "$\supseteq$". Let $w \in K(a)$ and $x \diamond w = u$. If $\neg F(x, w)$, then $u = \langle x, w\rangle$ and (α) yields $\theta_a(x, w)$. Otherwise $w = x^{[k-1]}s$ and $D(x, s) = \{u\}$. There are two cases. *(i)* $w = \langle c, a\rangle$ is generic. Comparing $\langle c, a\rangle = \langle x, x^{[k-2]}s\rangle$ gives $c = x$ and $a = x^{[k-2]}s$. Since $k - 2 \ge 1$, a is a pair with $a_L = x$, and $\neg F(a_L, a)$ because w is generic. So (β) yields a_L, which equals $\theta_a(x, w)$ for both types, since $x = a_L = c = o_a(w)$. *(ii)* $w \in \kappa(a)$. Then (γ) yields $\theta_a(x, w)$; the guard holds by Remark 5.2.

"$\subseteq$". If (α) applies, then $w := u_R \in K(a)$ and $u_L \diamond w = \langle u_L, w\rangle = u$, so the output $\theta_a(u_L, w)$ is in the right-hand side. If (β) applies, then $w := \langle a_L, a\rangle$ is a generic key, since $\neg F(a_L, a)$. Also $w = a_L^{[k-1]}s$ with $u \in D(a_L, s)$, so $a_L \diamond w = u$ by uniqueness, and the output is $a_L = \theta_a(a_L, w)$. If (γ) applies, then $w := v \in \kappa(a)$ satisfies $x \diamond v = u$, and the output is $\theta_a(x, v)$.

The final formulas follow from (a) and (b), since by (a) the keys of a are exactly the elements $c \diamond a$, with owner c. □

Remark 5.7 (Where $k \ge 3$ is essential, I). In case (i) of "$\supseteq$" the hypothesis $k \ge 3$ forces the unlocking factor x to equal a_L, so it is determined by the key. For $k = 2$ the same analysis gives $a = s$ with x arbitrary. Clause (β) would then have to range over unboundedly many x, and Lemma 5.1 would fail.

Example 5.8 (The three clauses at work; $k = 3$, $\theta = \lambda$)**.** Let $X = \{g, h\}$. *Clause* (α). $g \diamond g = \langle g, g\rangle$ is a generic key of g, so $u = \langle g, \langle g, g\rangle\rangle = g \diamond (g \diamond g)$ has $D(g, u) = \{g\}$. The products $g \diamond u$ and $g \diamond \langle g, u\rangle$ are formal, and $g \diamond \langle g, \langle g, u\rangle\rangle = g$; this is the instance $x = y = z = g$ of E_3. *Clause* (β). Let $s = h \diamond (g \diamond g) = \langle h, \langle g, g\rangle\rangle$, so $D(g, s) = \{h\}$, and let $y = \langle g, s\rangle$. Then $g \diamond y = \langle g, y\rangle = g^{[2]}s$ is formal, and $g \diamond \langle g, y\rangle$ contracts to h. So with $x = z = g$ the core $x \diamond (z \diamond y)$ of E_3 equals h, and clause (β) gives $D(y, h) = \{g\}$. Indeed $y \diamond h$ and $y \diamond (y \diamond h)$ are formal, and $L^3_y(h) = g$. *Clause* (γ). Let $v = g^{[2]}s$, $t = v \diamond (g \diamond h) = \langle v, \langle g, h\rangle\rangle$ and $y' = h^{[2]}t$. Then $D(h, t) = \{v\}$, so $\kappa(y') = \{v\}$, i.e. $h \diamond y' = v$ is the special key of y'. With $x = g$ and $z = h$ the core is $g \diamond v = h$, clause (γ) gives $D(y', h) = \{g\}$, and $L^3_{y'}(h) = g$. These computations are reproduced by the program in Appendix A.

Lemma 5.9 (Vacuity)**.** *For every $a \in \mathcal{T}$:*

(a) $D(a, a) = \emptyset$*;*

(b) $D(a, w) = \emptyset$ *for every $w \in K(a)$.*

Proof. We show that no clause applies; only the conditions of the clauses are used.

(a) *Clause* (α). Here a is a pair and $a_R \in K(a)$. Since $a_R \sqsubset a$, Lemma 5.3(c) gives $a_R \in \kappa(a)$. So $a = a_L^{[k-1]}t$ and $a_R \in D(a_L, t) \subseteq \mathrm{Sub}^-(a_L) \cup \mathrm{Sub}^-(t)$. But $a_R = a_L^{[k-2]}t$ with $k - 2 \geq 1$, so a_R properly contains t and is larger than a_L; it is in neither set. *Clause* (β). We would need $a \in D(a_L, s)$ with $a_L, s \sqsubset a$, which contradicts Lemma 5.1. *Clause* (γ). We would need $a \in D(x, s)$ with $x, s \sqsubset v \sqsubset a$, which again contradicts Lemma 5.1.

(b) First let $w = \langle c, a\rangle$ be generic. Clause (α) would need $w_R = a \in K(a)$, which is excluded by Lemma 5.3(c). Clauses (β) and (γ) would need $w \in D(x', s)$ for some $x', s \sqsubset a$, hence $w \sqsubset a$ by Lemma 5.1. That is impossible, since $a \sqsubset w$.

Now let $w = v \in \kappa(a)$. Clause (α) would need $v = \langle x, w'\rangle$ with $w' \in K(a)$. Then $w' \sqsubset v \sqsubset a$, so $w' \in \kappa(a) = \{v\}$ by Lemma 5.3(c), contradicting $w' \sqsubset v$. Clause (β) needs $\neg F(a_L, a)$, but $\kappa(a) \neq \emptyset$ means $F(a_L, a)$. Clause (γ) would need $v = x^{[k-1]}s$ and $v \in D(x, s)$; then $v \sqsubset x$ or $v \sqsubset s$ by Lemma 5.1, although $x, s \sqsubset v$. □

Lemma 5.10 (Decoding without premature contraction)**.** *Let $x \in D(a, u)$. Then $L^j_a(u) = a^{[j]}u$ for $0 \leq j \leq k - 1$, and $L^k_a(u) = x$.*

Proof. Induction on j; the case $j = 0$ is trivial. Let $0 \leq j \leq k - 2$ and assume $L^j_a(u) = a^{[j]}u$. Then $L^{j+1}_a(u) = a \diamond a^{[j]}u$. Suppose, for a contradiction, that $F(a, a^{[j]}u)$. Then $a^{[j]}u = a^{[k-1]}t$ with $D(a, t) \neq \emptyset$, i.e. $u = a^{[m]}t$ with $m := k - 1 - j \geq 1$. In particular $u = \langle a, a^{[m-1]}t\rangle$ properly contains a.

Since $D(a, u) \neq \emptyset$, some clause applies. Clauses (β) and (γ) require $u \in D(x', s)$ for some $x', s \sqsubset a$, so $u \sqsubset a$ by Lemma 5.1. This is impossible. Hence (α) applies, and $u_R = a^{[m-1]}t \in K(a)$.

- If $m = 1$, then $u_R = t \in K(a)$ and $D(a, t) \neq \emptyset$, contradicting Lemma 5.9(b).
- If $m \geq 2$, then $u_R = \langle a, a^{[m-2]}t\rangle$ properly contains a, so by Lemma 5.3(c) it is a generic key $\langle c, a\rangle$. Comparing gives $c = a$ and $a^{[m-2]}t = a$. If $m \geq 3$ this says $\langle a, \cdots\rangle = a$, which is impossible. If $m = 2$ it says $t = a$, and then $D(a, a) \neq \emptyset$ contradicts Lemma 5.9(a).

Hence $\neg F(a, a^{[j]}u)$ and $L^{j+1}_a(u) = a^{[j+1]}u$. Finally, $L^k_a(u) = a \diamond a^{[k-1]}u$. Since $D(a, u) = \{x\}$ is non-empty, $F(a, a^{[k-1]}u)$ holds, and the product is x. □

Remark 5.11 (Where $k \geq 3$ is essential, II). Lemma 5.9(a) uses $k \geq 3$ to separate the tower a_R from the special key. Lemma 5.10 uses vacuity to exclude every premature contraction among the first $k - 1$ unlocking steps.

6 Proof of the main theorem and corollaries

Theorem 6.1. *For every $k \geq 3$ and every non-empty set X, $\mathcal{T}_k^\lambda(X) \models \mathsf{E}_k$ and $\mathcal{T}_k^\omega(X) \models \mathsf{G}_k$.*

Proof. Let $x, y, z \in \mathcal{T}$. *Type* λ. By Lemma 5.6(a), $w := z \diamond y \in K(y)$. Put $u := x \diamond w$. By Lemma 5.6(b), $x = \theta_y(x, w) \in D(y, u)$. By Lemma 5.10, $L_y^k(u) = x$, which is E_k. *Type* ω. By Lemma 5.6(a), $w := x \diamond y \in K(y)$ with $o_y(w) = x$. Put $u := z \diamond w$. By Lemma 5.6(b), $x = o_y(w) = \theta_y(z, w) \in D(y, u)$. By Lemma 5.10, $L_y^k(u) = x$, which is G_k. □

Definition 6.2 (Normal trees)**.** A tree $t \in \mathcal{T}$ is *normal* (for the fixed type θ) if no subtree $\langle s, s' \rangle$ of t, including t itself, satisfies $F(s, s')$. Let $\mathcal{N} = \mathcal{N}(X)$ be the set of normal trees.

Lemma 6.3. *$X \subseteq \mathcal{N}$; $\mathcal{N}$ is closed under subtrees and under $\diamond$; every $\langle s, s' \rangle \in \mathcal{N}$ equals $s \diamond s'$. Hence $\mathcal{N}$ is the submagma of $(\mathcal{T}, \diamond)$ generated by X.*

Proof. Closure under subtrees is clear. Let $a, p \in \mathcal{N}$. If $\neg F(a, p)$, then $a \diamond p = \langle a, p \rangle$, and every subtree of $\langle a, p \rangle$ is a subtree of a, a subtree of p, or $\langle a, p \rangle$ itself, none of which fires. If $F(a, p)$, then $a \diamond p \in \mathrm{Sub}^-(p) \subseteq \mathcal{N}$ by (3). If $\langle s, s' \rangle \in \mathcal{N}$ then $\neg F(s, s')$, so $s \diamond s' = \langle s, s' \rangle$. By induction on size, every normal tree is a $\diamond$-word in elements of X. □

Theorem 6.4 (Freeness)**.** *Let $k \geq 3$, let $\mathsf{L} = \mathsf{E}_k$ if $\theta = \lambda$ and $\mathsf{L} = \mathsf{G}_k$ if $\theta = \omega$. For every model $(M, \bullet)$ of L and every map $f \colon X \to M$ there is a unique homomorphism $\varphi \colon (\mathcal{N}(X), \diamond) \to (M, \bullet)$ extending f. Thus $\mathcal{N}(X)$ is the free model of L on X.*

Proof. Uniqueness follows from Lemma 6.3. For existence, let $\varphi \colon \mathcal{T} \to M$ be the structural evaluation: $\varphi(g) = f(g)$ and $\varphi(\langle s, t \rangle) = \varphi(s) \bullet \varphi(t)$. We show $\varphi(a \diamond p) = \varphi(a) \bullet \varphi(p)$ for all $a, p \in \mathcal{N}$, by induction on $|a| + |p|$. If $\neg F(a, p)$, then $a \diamond p = \langle a, p \rangle$ and there is nothing to prove. Otherwise $p = a^{[k-1]}u$, $D(a, u) = \{x\}$ and $a \diamond p = x$. By the structural definition, $\varphi(a) \bullet \varphi(p) = L_{\varphi(a)}^k(\varphi(u))$, where now $L_m(t) = m \bullet t$. The clause that produces x provides trees x', w with $w \in K(a)$, $x' \diamond w = u$ and $x = \theta_a(x', w)$: in (α), $(x', w) = (u_L, u_R)$; in (β), $(x', w) = (a_L, \langle a_L, a \rangle)$; in (γ), $w = v \in \kappa(a)$ and $x' = v_L$. Put $c := o_a(w)$. We claim that

$$\varphi(u) = \varphi(x') \bullet \varphi(w) \quad \text{and} \quad \varphi(w) = \varphi(c) \bullet \varphi(a). \tag{4}$$

Given (4), the law L in M yields $L_{\varphi(a)}^k\big(\varphi(x') \bullet (\varphi(c) \bullet \varphi(a))\big) = \varphi(x')$ if $\theta = \lambda$, and $= \varphi(c)$ if $\theta = \omega$. In both cases this is $\varphi(\theta_a(x', w)) = \varphi(x)$.

To prove (4), note that all trees below are normal, being subtrees of a or p, or formal pairs of normal trees that do not fire. Note also that $|p| = (k-1)(|a|+1) + |u| \geq 2|a| + 2 + |u|$.

- (α) $u = \langle x', w \rangle$, so the first part holds structurally. If $w = \langle c, a \rangle$ is generic, the second part holds structurally too. If $w \in \kappa(a)$, then $w = a_L \diamond a$ with $c = a_L$ and $|a_L| + |a| < |a| + |p|$, so the second part holds by induction.

- (β) $x' = a_L$, $a = x'^{[k-2]}s$, and $w = \langle x', a \rangle$, which is normal since $\neg F(x', a)$. Then $x' \diamond w = u$ with $|x'| + |w| = 2|a_L| + |a| + 1 < 3|a| + 2 \leq |a| + |p|$, so the first part holds by induction. The second part holds structurally, with $c = x'$.

- (γ) $w = v = x'^{[k-1]}s \in \kappa(a)$, so $x' \diamond w = u$ and $w = a_L \diamond a$, with $c = a_L$. Here $|x'| + |w| < 2|a| < |a| + |p|$ and $|a_L| + |a| < |a| + |p|$, so both parts hold by induction.

In every case $\varphi(a \diamond p) = \varphi(x) = \varphi(a) \bullet \varphi(p)$. Finally, $\mathcal{N}(X)$ is a model of L by Theorem 6.1 and Lemma 6.3. □

Proof of Theorem 1.2. Combine Theorems 6.1 and 6.4. □

Proof of Corollary 1.3. $\mathcal{T}(X)$ is infinite and non-trivial; for instance $g \neq \langle g, g\rangle$ for $g \in X$. By Theorem 6.1 it carries a model of E_k and a model of G_k, and by Proposition 2.1(c) neither law has a non-trivial finite model. For $k = 3$ these are Equations 5093 and 5107. Reversing every product turns 5093 into 41179 and 5107 into 40951. Hence the opposite magmas $a \diamond^{\mathrm{op}} b := b \diamond a$ of $\mathcal{T}_3^{\lambda}(X)$ and $\mathcal{T}_3^{\omega}(X)$ satisfy 41179 and 40951, and the finite models of these dual laws are opposites of finite models of 5093 and 5107, hence trivial. □

Proof of Corollary 1.4. By Theorem 6.4, $s \approx t$ is a consequence of the law if and only if s and t have the same value in $\mathcal{N}(V)$ under the identity assignment on the variables. These values are computable: Definition 4.1 is a recursion over finitely many smaller pairs, so D, F and $\diamond$ are computable. □

Remark 6.5 (Two concrete consequences). Since the generators x and y are different normal trees, neither law implies $x \approx y$. Similarly, $x \diamond x = \langle x, x\rangle \neq x$ in $\mathcal{N}(\{x\})$, so neither law implies idempotency, and the free algebra on one generator is infinite. Many other non-implications can be read off the normal forms in the same way.

7 Discussion

7.1 How the construction was found

The construction came out of a forcing ("greedy") simulation for (E5093). We built partial models by repeatedly choosing triples (x, y, z), defining any missing product along the chain $z \diamond y$, $x \diamond (z \diamond y)$, ... as a fresh element, and adding the forced product $L_y^3(\cdots) = x$. Over more than 10^5 steps, including adversarial choices that complete chains through forced products, no conflict ever arose. In every chain we inspected (more than 3000 forced products), the two middle unlocking steps $y \diamond u_1$ and $y \diamond u_2$ were fresh pairs. This is exactly the no-premature-contraction phenomenon of Lemma 5.10, and it led directly to Definition 5.5. The owner variant for (E5107) was obtained by changing only the output map θ.

Remark 7.1 (Knuth–Bendix completion). Running Knuth–Bendix completion [8] with a size-based order on the single rule $y \diamond (y \diamond (y \diamond (x \diamond (z \diamond y)))) \to x$ produces, among others, the rules

$$W \diamond (W \diamond (W \diamond x_1)) \to x_0 \quad \text{with } W = x_0 \diamond (x_1 \diamond (x_2 \diamond x_0))$$

and

$$V \diamond (V \diamond (V \diamond x_0)) \to W \quad \text{with } V = W \diamond x_1.$$

These are instances of the identity $L_{a\diamond t}^3\big(L_a^3(t)\big) = a$, which is itself the instance $(x, y, z) = (a,\ a\diamond t,\ a)$ of (E5093). Our implementation generated more than fifty rules without terminating. Theorem 6.4 takes the place of this growing rule set. Every term is equivalent to exactly one normal tree, viewed as a term, and that tree is computed by the recursion of Definition 4.1.

7.2 The decoder-tree principle

The proofs use only a few features of the laws. Both have the shape $x \approx L_y^k(\Phi(x, y, z))$, where Φ "locks" x using a right multiple of y: in E_k, x is multiplied by a key of y; in G_k, x is the owner of a key of y. The proof has three parts:

(1) the firing pattern is the generic shape of the last unlocking step;

(2) the firing condition is the semantic predicate "the core lies in the image of $\Phi(\cdot, a, \cdot)$", unfolded into finitely many size-decreasing clauses, with negation used only at strictly smaller arguments;

(3) the subterm, uniqueness and vacuity lemmas exclude premature firing.

We expect the same scheme to apply to further laws of this shape, and many of the order-five laws that remain unclassified [5, 9] have this flavour. We make no claim about specific laws other than the families E_k and G_k.

Question 7.2. Which of the remaining unclassified order-five laws of [5] have free algebras with decoder-tree normal forms? Is there a decidable syntactic criterion on Φ for the scheme above to succeed?

Acknowledgements

We thank the contributors to the Equational Theories Project, whose blueprint, data and formalisations made the open problems precise and easy to check, and the authors of [9] for publishing their complete result files.

References

[1] A. K. Austin, *A note on models of identities*, Proc. Amer. Math. Soc. **16** (1965), 522–523.

[2] A. K. Austin, *Finite models for laws in two variables*, Proc. Amer. Math. Soc. **17** (1966), 1410–1412.

[3] E. Xie, W. Ye and A. Zhang, *ALPS: Measuring valid creativity in large language models with mathematical construction*, arXiv:2608.15979 (2026).

[4] M. Bolan et al. (Equational Theories Project), *The Equational Theories Project: Advancing collaborative mathematical research at scale*, arXiv:2512.07087 (2025).

[5] Equational Theories Project, *Blueprint*, in particular the chapters "Finite implications and infinite models", "Order 5 Austin laws" and "Equation 677", `https://teorth.github.io/equational_theories/blueprint/` (accessed October 2026).

[6] A. Kisielewicz, *Varieties of algebras with no nontrivial finite members*, in: Lattices, Semigroups, and Universal Algebra (Lisbon, 1988), Plenum, New York, 1990, 129–136.

[7] A. Kisielewicz, *Austin identities*, Algebra Universalis **38** (1997), no. 3, 324–328.

[8] D. E. Knuth and P. B. Bendix, *Simple word problems in universal algebras*, in: Computational Problems in Abstract Algebra (J. Leech, ed.), Pergamon, Oxford, 1970, 263–297.

[9] J. Zhao, B. Wu, T. Yang and X. Miao, *Trace-tree magmas: proof-producing infinite countermodels and 28 new order-five Austin classifications*, arXiv:2609.05690 (2026); result files at `https://github.com/YanbiaoLab/trace-tree-magmas`.

A Verification program

The program below is self-contained (Python 3, standard library only). It implements Definitions 4.1 and 5.5 with hash-consed trees for both types and for each $k \in \{3, 4, 5\}$, and checks, for each of these six magmas:

(i) the law (E_k for type λ, G_k for type ω) for all $102^3 = 1{,}061{,}208$ triples of trees with at most three internal nodes over two generators;

(ii) $2 \cdot 10^4$ random triples drawn from a pool closed under the operations that occur in the law;

(iii) 10^4 targeted instances in which the core of the law contracts through clause (β) or clause (γ), as in Example 5.8;

(iv) on $2 \cdot 10^4$ random instances: Lemma 5.6 (including owners), the subterm property, uniqueness, vacuity, the absence of premature contraction, and closure of normal trees under $\diamond$.

The exhaustive families in (i) mainly exercise the generic path. For $k = 3$ only 8 of the 1,061,208 triples (for each type) have a contracting core, and for $k \geq 4$ none do. The evidence for clauses (β) and (γ) therefore comes from (iii) and (iv). For $k = 3$ and each type, the program also compares the clause-based operation with an independent implementation of the semantic description of Lemma 5.6, in which $D(a, u)$ is computed by exhaustive search over candidate pairs. The two agree on all sampled products, including firing products, and the independent implementations satisfy E_3 and G_3 on all 22^3 triples of trees with at most two internal nodes. The full run takes about 10 minutes and reports `ALL CHECKS PASSED`. The source of this manuscript is a single file: compiling it writes the program to `verify_decoder_trees.py` (via `filecontents`), and the listing below is read back from that file. During development, further runs checked 5.7×10^6 triples (y with at most four internal nodes) for E_3, and the laws for $k \leq 6$; all passed. These computations play no role in the proofs.

```
#!/usr/bin/env python3
"""Computational companion to "Decoder Trees and Kisielewicz's Identity:
Equations 5093 and 5107 Are Austin Laws".

For k >= 3 and theta in {'left', 'owner'} this implements the decoder-tree magma
T_k^theta(X) of the paper (Definitions 4.1 and 5.5) and checks the law
    theta = 'left' :  E_k : x = y*(y*( ... (y*(x*(z*y))) ... ))   (k leading y's)
    theta = 'owner':  G_k : x = y*(y*( ... (y*(z*(x*y))) ... ))   (k leading y's)
on large finite samples (E_3 is ETP Equation 5093 and G_3 is ETP Equation 5107).
It also checks every structural lemma of the paper on random and targeted
instances, and cross-checks the clause-based implementation against an
independent brute-force implementation of the semantic description (Lemma 5.6).
Usage:  python3 verify_decoder_trees.py [--quick]   (standard library only; each
family/k is checked in a separate process to bound memory use)
"""
import sys, random
from functools import lru_cache
sys.setrecursionlimit(1 << 20)

# ---------- hash-consed binary trees: node id -> None (leaf) or (left, right) ----------
NODES, IDX, SIZE = [], {}, []
def leaf(i):
    key = ('leaf', i)
    if key not in IDX:
        IDX[key] = len(NODES); NODES.append(None); SIZE.append(1)
    return IDX[key]
def pair(l, r):
    key = (l, r)
    if key not in IDX:
        IDX[key] = len(NODES); NODES.append(key); SIZE.append(SIZE[l] + SIZE[r] + 1)
    return IDX[key]
def ispair(t): return NODES[t] is not None
def L(t): return NODES[t][0]
def R(t): return NODES[t][1]
def tower(c, j, u):                       # c^[j] u = <c,<c,...<c,u>...>>
    for _ in range(j): u = pair(c, u)
    return u
def untower(c, p, j):                     # the t with p = c^[j] t, or None
    for _ in range(j):
        if not ispair(p) or L(p) != c: return None
        p = R(p)
    return p
def proper_subtrees(t):
```

```
    out, st = set(), [t]
    while st:
        s = st.pop()
        if ispair(s):
            for c in (L(s), R(s)):
                if c not in out: out.add(c); st.append(c)
    return out

class DecoderTree:
    """The decoder-tree magma T_k^theta (Definitions 4.1 and 5.5 of the paper)."""
    def __init__(self, k, theta):
        assert k >= 3 and theta in ('left', 'owner')
        self.k, self.theta = k, theta
        self.D = lru_cache(maxsize=None)(self._D)
        self.F = lru_cache(maxsize=None)(self._F)
        self.kappa = lru_cache(maxsize=None)(self._kappa)
        self.mul = lru_cache(maxsize=None)(self._mul)
        self.nonunique = []
    def _F(self, c, p):                  # F(c,p): p = c^[k-1] t with D(c,t) nonempty
        t = untower(c, p, self.k - 1)
        return t is not None and len(self.D(c, t)) > 0
    def _kappa(self, a):                 # special key set: D(a_L, t) if a = a_L^[k-1] t
        if not ispair(a): return frozenset()
        t = untower(L(a), a, self.k - 1)
        return self.D(L(a), t) if t is not None else frozenset()
    def generic_key(self, a, w):         # w = <c,a> with not F(c,a)
        return ispair(w) and R(w) == a and not self.F(L(w), a)
    def is_key(self, a, w):              # w in K(a)
        return self.generic_key(a, w) or w in self.kappa(a)
    def owner(self, a, w):               # o_a(w): c for w = <c,a> generic, a_L for the special key
        return L(w) if self.generic_key(a, w) else L(a)
    def out(self, a, x, w):              # theta_a(x, w)
        return x if self.theta == 'left' else self.owner(a, w)
    def _D(self, a, u):
        k, res = self.k, set()
        if ispair(u) and self.is_key(a, R(u)) and not self.F(L(u), R(u)):        # clause alpha
            res.add(self.out(a, L(u), R(u)))
        if ispair(a):                                                            # clause beta
            s = untower(L(a), a, k - 2)
            if s is not None and not self.F(L(a), a) and u in self.D(L(a), s):
                res.add(L(a))
        for v in self.kappa(a):                                                  # clause gamma
            if ispair(v):
                x = L(v); s = untower(x, v, k - 1)
                if s is not None and SIZE[x] + SIZE[s] < SIZE[a] + SIZE[u] and u in self.D(x, s):
                    res.add(self.out(a, x, v))
        if len(res) > 1: self.nonunique.append((a, u, res))
        return frozenset(res)
    def _mul(self, a, p):
        t = untower(a, p, self.k - 1)
        if t is not None:
            d = self.D(a, t)
            if d: return min(d)
        return pair(a, p)
    def Lpow(self, a, u, n):
        for _ in range(n): u = self.mul(a, u)
        return u
    def core(self, x, y, z):             # the term inside L_y^k
        return self.mul(x, self.mul(z, y)) if self.theta == 'left' else self.mul(z, self.mul(x, y))
    def law(self, x, y, z):
        return self.Lpow(y, self.core(x, y, z), self.k) == x

def trees_upto(gens, max_internal):
    by = {0: list(gens)}
    for n in range(1, max_internal + 1):
        by[n] = [pair(l, r) for i in range(n) for l in by[i] for r in by[n - 1 - i]]
    return [t for n in range(max_internal + 1) for t in by[n]]

def structured_pool(M, base, steps, rng):
    pool, k = list(base), M.k
    for _ in range(steps):
        a, b, c = rng.choice(pool), rng.choice(pool), rng.choice(pool)
        r = rng.random()
```

```
        if r < 0.3: pool.append(M.mul(a, b))
        elif r < 0.6: pool.append(M.Lpow(a, M.core(b, a, c), rng.randrange(k + 1)))
        else:
            y = M.mul(a, b); pool.append(y)
            pool.append(M.Lpow(y, M.core(a, y, a), rng.randrange(k)))
    return pool

# ---------- independent brute-force implementation of the semantics (nested tuples) ----------
def bf_model(k, theta):
    def isleaf(t): return t[0] == 'g'
    def psub(t):
        out, st = set(), [t]
        while st:
            s = st.pop()
            if not isleaf(s):
                for c in s:
                    if c not in out: out.add(c); st.append(c)
        return out
    def untower_t(c, p, j):
        for _ in range(j):
            if isleaf(p) or p[0] != c: return None
            p = p[1]
        return p
    @lru_cache(maxsize=None)
    def mul(a, p):                        # a*p = x iff p = a^[k-1]u and dec(a,u) = {x}
        u = untower_t(a, p, k - 1)
        if u is not None:
            d = dec(a, u)
            if d:
                assert len(d) == 1
                return next(iter(d))
        return (a, p)
    @lru_cache(maxsize=None)
    def dec(a, u):                        # E: {x : x*(c*a) = u};  G: {x : z*(x*a) = u}, by search
        cand = psub(a) | psub(u) | {a, u}
        if theta == 'left':
            return frozenset(x for x in cand for c in cand if mul(x, mul(c, a)) == u)
        return frozenset(x for x in cand for z in cand if mul(z, mul(x, a)) == u)
    return mul

def to_tuple(t, memo={}):
    if t not in memo:
        memo[t] = ('g', [kk for kk, v in IDX.items() if v == t][0][1]) if not ispair(t) else (to_tuple(L(
     t)), to_tuple(R(t)))
    return memo[t]

def check_family(theta, k, quick, rng):
    M, gens = DecoderTree(k, theta), [leaf(0), leaf(1)]
    small = trees_upto(gens, 2 if quick else 3)
    name = ('E' if theta == 'left' else 'G') + str(k)
    ok = True
    bad = sum(1 for x in small for y in small for z in small if not M.law(x, y, z))
    print(f"[{name}] exhaustive: {len(small)**3} triples, violations {bad}", flush=True); ok &= bad == 0
    pool = structured_pool(M, small, 300 if quick else 1500, rng)
    N = 3000 if quick else 20000
    bad = sum(1 for _ in range(N) if not M.law(rng.choice(pool), rng.choice(pool), rng.choice(pool)))
    print(f"[{name}] structured random: {N} triples, violations {bad}", flush=True); ok &= bad == 0
    # targeted instances making the core contract through clause beta or clause gamma
    bad = n = 0
    for _ in range(300 if quick else 2000):
        x, q, c1, c, c2, xx, zz = (rng.choice(pool) for _ in range(7))
        s = M.core(q, x, c1)                    # q is in D(x, s)
        y1 = tower(x, k - 2, s)                 # beta: x * <x,y1> contracts
        v = tower(x, k - 1, s)
        t = M.core(v, c, c2)                    # v is in D(c, t)
        y2 = tower(c, k - 1, t)                 # kappa(y2) = {v}: gamma
        assert M.kappa(y2) == frozenset([v])
        inst = ((x, y1, x), (xx, y1, zz), (x, y2, c), (xx, y2, c), (xx, y2, zz)) if theta == 'left' else
     \
               ((x, y1, x), (xx, y1, zz), (c, y2, x), (c, y2, zz), (xx, y2, zz))
        for (a, b, cc) in inst:
            n += 1; bad += not M.law(a, b, cc)
    print(f"[{name}] targeted beta/gamma instances: {n}, violations {bad}", flush=True); ok &= bad == 0
```

```
    # lemma checks
    viol = dict(Sub=0, U=0, V1=0, V2=0, Sem=0, Dec=0, Normal=0)
    memo = {}
    def normal(t):
        if t not in memo:
            memo[t] = (not ispair(t)) or (normal(L(t)) and normal(R(t)) and not M.F(L(t), R(t)))
        return memo[t]
    for _ in range(3000 if quick else 20000):
        a, x, c, p = (rng.choice(pool) for _ in range(4))
        w = M.mul(c, a)                          # a key of a, with owner c
        u = M.mul(x, w) if theta == 'left' else M.mul(x, M.mul(c, a))
        expected = x if theta == 'left' else c
        d = M.D(a, u)
        viol['Sem'] += (not M.is_key(a, w)) or M.owner(a, w) != c or (expected not in d)
        viol['U'] += len(d) > 1
        viol['Sub'] += not d <= (proper_subtrees(a) | proper_subtrees(u))
        t, early = u, False
        for _j in range(k - 1):
            t2 = M.mul(a, t); early |= (t2 != pair(a, t)); t = t2
        viol['Dec'] += early or M.mul(a, t) != expected
        viol['V1'] += len(M.D(a, a)) > 0
        viol['V2'] += len(M.D(a, w)) > 0
        if normal(a) and normal(p): viol['Normal'] += not normal(M.mul(a, p))
    print(f"[{name}] lemma checks (violations): {viol}; non-unique D: {len(M.nonunique)}", flush=True)
    ok &= sum(viol.values()) == 0 and not M.nonunique
    return ok

def cross_check(theta, quick, rng):
    k = 3; M, bf = DecoderTree(k, theta), bf_model(k, theta)
    small = trees_upto([leaf(0), leaf(1)], 2)
    T = lambda t: to_tuple(t)
    def bf_law(x, y, z):
        core = bf(T(x), bf(T(z), T(y))) if theta == 'left' else bf(T(z), bf(T(x), T(y)))
        for _ in range(k): core = bf(T(y), core)
        return core == T(x)
    bad = sum(1 for x in small for y in small for z in small if not bf_law(x, y, z))
    pool = [p for p in structured_pool(M, small, 200, rng) if SIZE[p] < 120]
    dis = fires = 0
    for _ in range(600 if quick else 3000):
        a = rng.choice(pool); r = rng.random()
        if r < 0.4: p = rng.choice(pool)
        elif r < 0.7: p = tower(a, k - 1, rng.choice(pool))
        else: p = tower(a, k - 1, M.core(rng.choice(pool), a, rng.choice(pool)))   # a firing product
        res = M.mul(a, p); fires += res != pair(a, p)
        dis += T(res) != bf(T(a), T(p))
    name = 'E3' if theta == 'left' else 'G3'
    print(f"[cross-check {name}] brute-force semantic model: law violations {bad} on {len(small)**3}
     triples; "
          f"product disagreements {dis} ({fires} firing products compared)", flush=True)
    return bad == 0 and dis == 0

def example_check():
    """Reproduces Example 5.8 of the paper (k = 3, type lambda)."""
    M, g, h, P = DecoderTree(3, 'left'), leaf(0), leaf(1), pair
    u = M.mul(g, M.mul(g, g))                                  # clause alpha
    assert u == P(g, P(g, g)) and M.D(g, u) == {g} and M.Lpow(g, u, 3) == g
    s = M.mul(h, M.mul(g, g)); y = P(g, s)                     # clause beta
    assert s == P(h, P(g, g)) and M.D(g, s) == {h}
    assert M.mul(g, y) == P(g, y) and M.mul(g, P(g, y)) == h and M.D(y, h) == {g}
    assert M.mul(y, h) == P(y, h) and M.mul(y, P(y, h)) == P(y, P(y, h)) and M.Lpow(y, h, 3) == g
    v = tower(g, 2, s); t = M.mul(v, M.mul(g, h)); y2 = tower(h, 2, t)   # clause gamma
    assert t == P(v, P(g, h)) and M.D(h, t) == {v} and M.kappa(y2) == {v}
    assert M.mul(h, y2) == v and M.mul(g, v) == h and M.D(y2, h) == {g} and M.Lpow(y2, h, 3) == g
    print("[example] Example 5.8 reproduced", flush=True)
    return True

def run_one(job, quick):
    theta, k = job[0], int(job[1])
    rng = random.Random(20261028 + 10 * k + (theta == 'owner'))
    if k == 0: return cross_check(theta, quick, rng)
    ok = example_check() if (theta, k) == ('left', 3) else True
    return check_family(theta, k, quick, rng) and ok
```

```
def main(quick):
    # each job runs in a fresh process, so that memory use stays bounded
    import subprocess
    ok = True
    for theta in ('left', 'owner'):
        for k in (3, 4, 5, 0):                    # k = 0 means: cross-check for k = 3
            cmd = [sys.executable, __file__, '--one', theta, str(k)] + (['--quick'] if quick else [])
            ok &= subprocess.run(cmd).returncode == 0
    print("ALL CHECKS PASSED" if ok else "SOME CHECK FAILED")
    return 0 if ok else 1

if __name__ == '__main__':
    quick = '--quick' in sys.argv
    if '--one' in sys.argv:
        i = sys.argv.index('--one')
        sys.exit(0 if run_one(sys.argv[i + 1:i + 3], quick) else 1)
    sys.exit(main(quick))
```